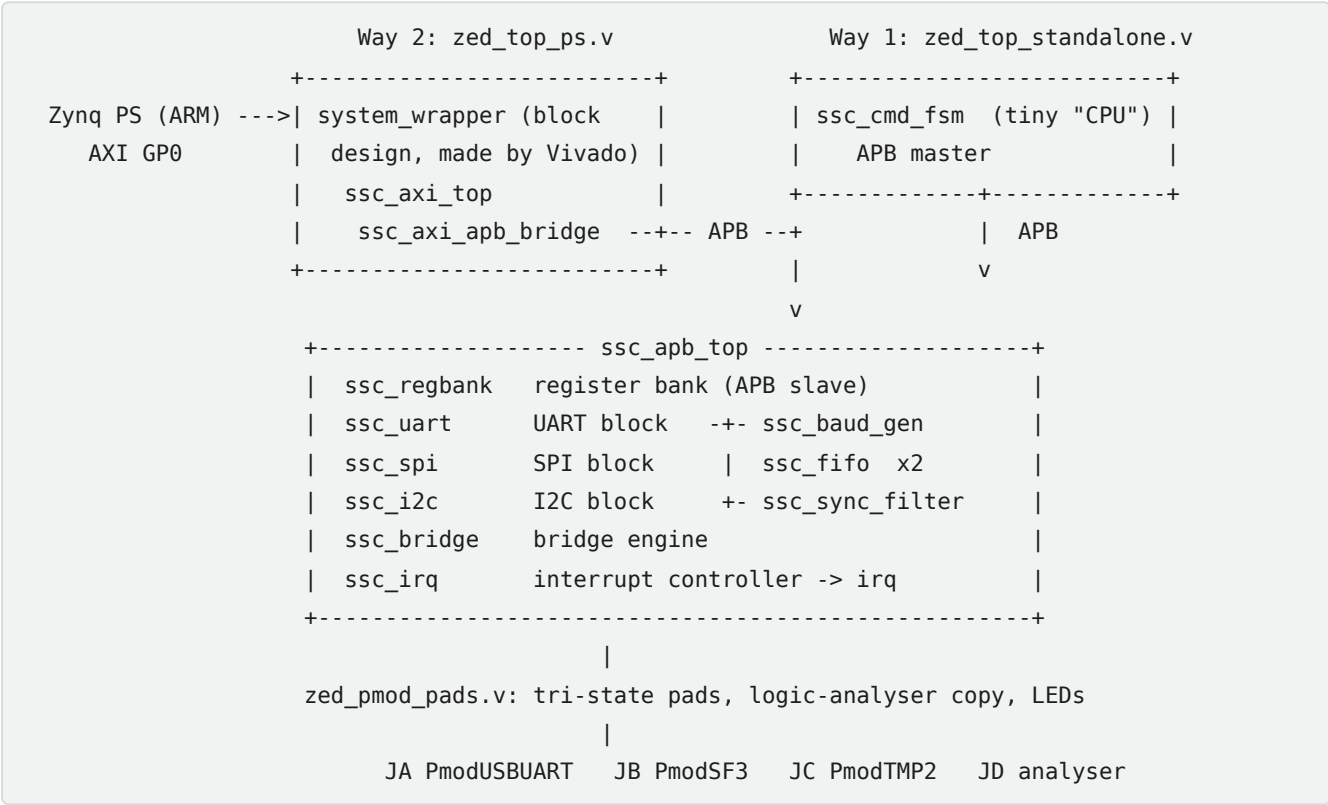


Code walkthrough

This page explains every Verilog file, in the order the data flows through it. Each file also starts with a comment block that says the same thing in short.

1. How the files map to the architecture slide



The same `ssc_apb_top` is used in both ways. Only what sits on top of it changes: in Way 1 it is a state machine, in Way 2 it is the ARM.

All files are plain Verilog-2001 and run in Vivado, XSim and Icarus Verilog. Each protocol block is built from the same five parts the slides list:

Part	File	Job
clock generator	<code>ssc_baud_gen.v</code> (UART); a counter inside SPI and I2C	sets the speed
control FSM	inside each block	the protocol steps
shift register	inside each block	turns bytes into bits and back
TX and RX FIFOs	<code>ssc_fifo.v</code>	queue 16 entries each way
glitch filter	<code>ssc_sync_filter.v</code>	cleans every input pin

2. The shared building blocks

ssc_sync_filter.v : synchroniser and glitch filter

The pins change at random moments compared with our 100 MHz clock. The module deals with this in three steps:

```
sync <= {sync[0], din};           // 2 flip-flops: kill metastability
hist <= {hist[FILTER_LEN-2:0], sync[1]}; // remember the last N samples
if (&hist)      dout <= 1'b1;      // all ones -> clean 1
else if (~|hist) dout <= 1'b0;    // all zeros -> clean 0
```

1. The first flip-flop can go "metastable": stuck halfway between 0 and 1 for a moment. The second flip-flop gives it a whole clock period to settle.
2. A shift register keeps the last N samples.
3. The output changes only when all N samples agree. Any spike shorter than N clocks (40 ns with N = 4) disappears.

`rise` and `fall` are one-clock pulses on the edges of the clean signal. The I2C bus monitor and the SPI slave use them.

ssc_fifo.v : 16-entry FIFO

- A memory array, a write pointer, a read pointer and a `count`.
- `rd_data = mem[rptr]` is read **without** a clock. This is called first-word fall-through: the oldest entry is always visible, and `rd_en` only throws it away. The register bank and the bridge can therefore read and pop in the same clock.
- `empty = (count == 0)` and `full = (count == 16)`.
- A write to a full FIFO, or a read from an empty one, is ignored. The blocks around the FIFO turn these into "overflow" events.
- Vivado builds the memory from LUT-RAM because it has no reset and is read without a clock. Each FIFO costs a few LUTs.

ssc_baud_gen.v : fractional baud generator (slide 10)

The UART needs a tick at $16 \times \text{baud}$. At 115200 baud: $100 \text{ MHz} / (16 \times 115200) = 54.25$.

```
wire [16:0] last      = div - 1 + extra; // period is 54 or 55 clocks
wire [4:0] acc_next = acc + div_frac;    // add 4/16 every tick
...
acc  <= acc_next[3:0];
extra <= acc_next[4]; // carry -> next period is 55
```

The fraction (4 sixteenths) goes into a 4-bit accumulator at every tick. Every fourth tick the accumulator overflows and the next period gets one extra clock: 54, 54, 54, 55, 54, ... The average is exactly 54.25.

Divider	Error at 115200 baud	Error at 460800 baud
Plain whole-number counter	0.47 %	3.2 % (a UART fails above about 2 %)
This fractional divider	0.006 %	0.006 %

The testbench measures these errors (test [9]).

3. `ssc_uart.v`: UART block

```
TX FIFO --> TX engine -----> txd
                        start | d0 d1 .. d(n-1) | parity | stop [stop]
rxd --> sync_filter --> RX engine (16 samples per bit, vote on 7,8,9) --> RX FIFO
```

TX engine. The states are `TX_IDLE`, `TX_START`, `TX_DATA`, `TX_PAR` and `TX_STOP`. Every state lasts 16 ticks (`tx_tick` counts 0..15).

- `tx_load` starts a frame when all four conditions hold: the TX FIFO has a byte, TX is enabled, CTS allows it, and we are idle or at the very end of a stop bit.
- Starting at the end of a stop bit lets frames follow each other with no gap.
- Parity is $\text{^(data \& mask) \^ parity_odd}$: an XOR of all the data bits, and inverted for odd parity.

RX engine.

1. In `RX_IDLE`, a low line at a tick counts as sample 0 of a start bit.
2. In every bit, samples 7, 8 and 9 (the middle of the bit) are collected. `vote` keeps the majority of the three:

```
wire vote = (v0 & v1) | (v0 & rx_s) | (v1 & rx_s); // samples 7, 8 and 9
```

3. If the middle of the start bit is 1, it was a glitch and the engine goes back to idle. Test [8] checks this.
4. Data bits are shifted in from the top, `rx_shift <= {vote, rx_shift[7:1]}`, and then lined up: `rx_word = rx_shift >> (8 - nbits)`.
5. In the middle of the stop bit:
 - Stop bit = 1: the byte goes into the RX FIFO, a parity check runs, and an overflow event fires if the FIFO is full.
 - Stop bit = 0: it is a framing error. If all the bits were 0, it is a **break** instead.
 - In both stop-bit-0 cases, the engine waits for the line to go high again.

Flow control. `rts_n` goes high (meaning "stop sending") when 14 or more bytes are waiting. A new frame starts only while `cts_n` is low.

Loopback. TX is connected to RX inside the chip, and the pin stays idle. The self-tests use this mode.

4. `ssc_spi.v`: SPI block, master and slave

Master engine

```
M_IDLE -> M_SETUP -> M_XFER (2N edges) -> M_HOLD -> next word? -> M_SETUP
\-> M_GAP (CS# high) -> M_IDLE
```

- `m_cnt` counts one **half** SCLK period: `clk_div + 1` clocks. Each time it wraps (`m_half`), SCLK toggles. A word of N bits needs 2N edges. Edges 0, 2, 4, ... are *leading* and edges 1, 3, 5, ... are *trailing*.
- The four SPI modes come down to one rule:

CPHA	Leading edge	Trailing edge
0	sample MISO	put the next bit on MOSI
1	put the next bit on MOSI	sample MISO

CPOL only sets the idle level of SCLK (`sclk <= cpol`).

- Words of 4 to 32 bits and MSB-first or LSB-first order are handled by five small functions:
 - `tx_align` moves the first bit to the head of the shifter.
 - `tx_head` reads that bit.
 - `tx_shift` brings the next bit to the head.
 - `rx_shift` shifts a received bit in.
 - `rx_final` lines up the received word at bit 0.
- Chip select works in one of two ways:
 - **Automatic:** `m_cs` stays low while words keep coming.
 - **Manual:** software holds CS# low with `CS_LEVEL`. Flash commands need this because they are longer than one FIFO-full.

The `cs_n` outputs are registered so they never glitch.

- **Abort:** when `m_run` drops, the engine goes back to idle at once, releases CS# and returns SCLK to idle. The TX FIFO is flushed in the same clock. Test [14] checks this.
- MISO goes through a 2-flip-flop synchroniser. That is why `clk_div` is at least 3, which gives a maximum SCLK of 12.5 MHz.

Slave engine

The outside master's SCLK, MOSI and CS# go through `ssc_sync_filter`. The engine reacts to the `rise` and `fall` pulses of SCLK:

1. When CS# falls, the first reply word is loaded from the TX FIFO.
2. The rule above, with "leading" and "trailing" swapped around, decides when to sample MOSI and when to change MISO.
3. After N bits, the word goes into the RX FIFO.
4. When CS# rises, the `SPI_DONE` event fires.

Because the edges are detected inside our 100 MHz domain, the outside SCLK should be about 4 MHz or slower.

5. `ssc_i2c.v`: I2C master block

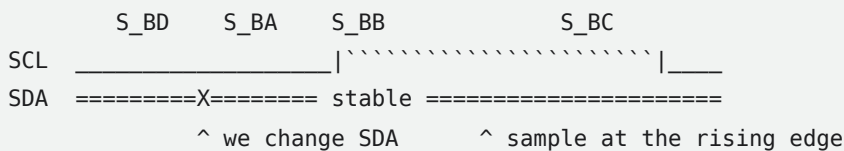
Open drain

Nobody ever drives a 1 on an I2C line. Our outputs are therefore "pull low" signals:

```
assign scl_oe = scl_low; // in zed_pmod_pads.v: assign pad = oe ? 1'b0 : 1'bz;
```

The pad becomes an IOBUF, and the pull-up resistors make the 1. Because several devices can pull low, the bus behaves as a wired AND. This is what makes clock stretching and arbitration possible.

One bit = four quarter periods



`qcnt` counts one quarter: `prescale + 1` clocks. Then:

- **Clock stretching:** in `S_BB` we let go of SCL, but the high time only starts counting when SCL is **seen** high (`high_seen`). A slow slave can hold SCL low as long as it likes. Test [21] checks this.
- **Clock synchronisation:** if SCL goes low early during the high phase because another master pulled it, we jump to `S_BD` and follow.
- **Arbitration:** if we let SDA go high but read 0 at the rising edge, another master is sending a 0. We have lost:

```
if (sending && my_bit && !sda_s) lose; // release both lines, raise ARB_LOST
```

Test [23] races a second master against the controller.

Transactions

- `stage` steps through the address bytes:
 - 7-bit address: `{addr, R/W}`, then the data bytes.
 - 10-bit address: `11110 A9 A8 0`, then `A7..A0`. For a read this is followed by a **repeated START** and `11110 A9 A8 1`.
- `remaining` counts the data bytes left. On a read, the controller sends ACK after every byte except the last, which gets a NACK. `my_bit` encodes all of this.
- When the slave does not ACK, the controller sends STOP and raises `NACK` and `DONE`.

- When `STOP = 0` in the command, the controller does not release the bus. It parks in `S_HOLD` with SCL low, and the next command starts with a repeated START. This is exactly the write-then-read of the temperature sensor (slide 13).

- **FIFO stalls:**

- TX FIFO empty in the middle of a write: wait in `S_WTX`.
- RX FIFO full in the middle of a read: wait in `S_WRX`.

Both waits hold SCL low, which I2C allows. Test [22] reads 20 bytes through the 16-entry FIFO this way.

- **Bus monitor:** a START seen on the bus (SDA falls while SCL is high) sets `bus_busy`, and a STOP clears it. A command waits (`pending`) while another master owns the bus. Test [24] checks this.

6. `ssc_bridge.v`: bridge engine (the base paper's idea)

The bridge has two channels, each with a 2-bit source and a 2-bit destination. The codes are the paper's COSE codes: 1 = SPI, 2 = I2C, 3 = UART.

```
wire ok0 = src0 && dst0 && !rx_empty[src0] && !tx_full[dst0];
wire go0 = ~stall & ok0 & (~ok1 | ~turn);      // channels take turns
...
if (go0) begin rx_pop[src0] = 1; tx_push[dst0] = 1; tx_data = pick(src0); end
```

In one clock, a byte leaves one block's RX FIFO and enters another block's TX FIFO, with no CPU involved.

- Two channels give a **two-way** bridge, which was the base paper's future work. Test [26] runs UART ↔ SPI.
- When the source equals the destination, you get the paper's pass-through, for example a hardware echo. Test [25] checks this.
- The bridge waits (`stall`) while the CPU is on the bus, so the two never touch a FIFO in the same clock.
- I2C needs an address for every transaction, so when I2C is the destination the block runs in `AUTO_WR` mode: every byte becomes a 1-byte write to `I2C_ADDR`. Test [27] checks this.

7. `ssc_irq.v`: interrupt controller

```
next  = (status & ~clear) | events;  // sticky flags; a new event beats a clear
irq   <= |(next & enable);           // one line to the ARM (IRQ_F2P[0] = ID 61)
```

8. `ssc_regbank.v`: register bank (APB slave)

APB takes two clocks per transfer:

- **Setup:** `PSEL` = 1.
- **Access:** `PSEL` = 1 and `PENABLE` = 1. The register is written or read at the end of this clock.

```

wire wr = psel & penable & pwrite;
wire rd = psel & penable & ~pwrite;
assign uart_tx_push = wr & (a == A_UART_TX);           // write -> FIFO
assign uart_rx_pop  = rd & (a == A_UART_RX) & ~uart_status[2]; // read -> pop

```

- Settings registers are ordinary flip-flops, written in one `case`.
- Commands are one-clock strobes taken straight from `pwdata` bits. Examples are the `_FLUSH`, `ABORT` and `I2C_CMD` bits.
- Reads go through a combinational multiplexer.
- Reading a data register pops the FIFO in the same access. When the FIFO is empty, bit 31 says "empty" and nothing is popped.
- The full register map is in [REGISTER_MAP.md](#).

9. `ssc_apb_top.v` : wiring

This file instantiates everything and shares each FIFO port between the CPU and the bridge:

```

wire u_tx_push = rb_u_tx_push | br_tx_push[3];
wire [7:0] u_tx_data = rb_u_tx_push ? pwdata[7:0] : br_tx_data;

```

It also packs every block's status into the common status layout and collects the 16 event lines for `ssc_irq`.

10. Bus front ends

- `ssc_axi_apb_bridge.v` (Way 2). This is an AXI4-Lite slave. It waits for a write (AW and W) or a read (AR), runs the APB setup and access phases, then returns B or R. It does the same job as the Xilinx AXI-APB Bridge box on the slide, but it is plain Verilog, so the whole path can be simulated (`tb_ssc_axi.v`).
- `ssc_axi_top.v` (Way 2). This puts the bridge and the controller together and adds the `X_INTERFACE_*` attributes, so that Vivado's block designer recognises `S_AXI`, its clock and reset, and the interrupt.
- `ssc_cmd_fsm.v` (Way 1). This is a tiny APB master that acts as the CPU:
 - `apb_call(write, address, data, return_state)` does one APB transfer and then jumps to the return state, like a function call.
 - `print(message, return_state)` sends a text message to `UART_TXDATA`, one character at a time, waiting whenever the TX FIFO is full. Messages are 128-byte strings, and zero bytes are skipped. That trick also hides the leading zeros of numbers.
 - The temperature goes from the raw bytes to text in three steps:
 1. `t13 = {msb, lsb} >> 3` (two's complement).
 2. The whole degrees go through **double dabble**, one shift per clock. This is binary-to-decimal conversion that never divides.
 3. The sixteenths come from a 16-entry table of exact decimals: 0.0625, 0.1250, and so on.

11. Board files

File	What it does
<code>zed_pmod_pads.v</code>	SPI pin directions for master or slave mode, I2C open-drain pads, the copy of every bus line on JD for a logic analyser, and LED activity stretchers
<code>zed_top_standalone.v</code>	Way 1. Power-on reset and BTNC reset, <code>ssc_cmd_fsm</code> , <code>ssc_apb_top</code> and the pads
<code>zed_top_ps.v</code>	Way 2. Wraps the Vivado-made <code>system_wrapper</code> (Zynq and controller) and adds the pads
<code>zed_pmods.xdc</code> , <code>zed_standalone.xdc</code>	Pin locations, 3.3 V I/O standards, pull-ups and timing exceptions

12. Software (`sw/`)

File	What it does
<code>ssc_regs.h</code>	The register map as C <code>#define</code> s. It must match <code>ssc_regbank.v</code> .
<code>ssc_driver.c</code>	Driver for the controller (details below)
<code>main.c</code>	A command menu on the ZedBoard's on-board USB-UART: <code>selftest</code> , <code>temp</code> , <code>id</code> , <code>flash</code> , <code>send</code> , <code>rx</code> , <code>bridge</code> , <code>bench</code> , <code>regs</code>

The driver in `ssc_driver.c` contains:

- **Register access:** `Xil_In32` and `Xil_Out32`.
- **Interrupt setup:** the GIC, interrupt 61, level-sensitive.
- **The ISR:** it collects event flags and moves UART bytes into a ring buffer.
- **Protocol helpers:** `ssc_uart_*`, `ssc_spi_xfer` (CS# held low for any length), `flash_*` for the PmodSF3, `ssc_i2c_xfer` (write, then a repeated START and a read), and `tmp2_read`.